

Hands on Numpy!

```
In [1]: import sys
import numpy as np
```

```
In [4]: np.array([1,6,0,45])
```

```
Out[4]: array([ 1,  6,  0, 45])
```

```
In [17]: a = np.array([1,6,0,45]) #creating numpy array & assigning to variable "a"
```

```
In [6]: b = np.array([0, 6.7, 1.6, 7.5, 72])
```

```
In [7]: a[3], b[-1]
```

```
Out[7]: (np.int64(45), np.float64(72.0))
```

```
In [16]: a[1],a[0] #getting by index
```

```
Out[16]: (np.int64(6), np.int64(1))
```

```
In [15]: a[:, b[0:], b[-1], b[-1:]] #starting from last element & after = only last elem
```

```
Out[15]: (array([ 1,  6,  0, 45]),
          array([ 0. ,  6.7,  1.6,  7.5, 72. ]),
          np.float64(72.0),
          array([72.]))
```

```
In [13]: b[1:-1] #Leaving first & last elements
```

```
Out[13]: array([6.7, 1.6, 7.5])
```

```
In [18]: a = np.array([10, 20, 30, 40, 50, 60])
print(a[::2])
```

```
[10 30 50]
```

```
In [19]: a[::-2] # starts from the end and moves backward by 2, resulting in [60, 40, 20]
```

```
Out[19]: array([60, 40, 20])
```

```
In [22]: a[0], a[2], a[-1] #getting by index
```

```
Out[22]: (np.int64(10), np.int64(30), np.int64(60))
```

Array Types

```
In [24]: b
```

```
Out[24]: array([ 0. ,  6.7,  1.6,  7.5, 72. ])
```

```
In [25]: b.dtype
```

```
Out[25]: dtype('float64')
```

```
In [26]: a.dtype
```

```
Out[26]: dtype('int64')
```

```
In [35]: c = np.array([1, 2, 3, 4], dtype=np.float64)
```

```
In [46]: print(c.itemsize) # Output: 8 (bytes per element)
         print(c.nbytes)   # Output: 32 (total bytes for all 4 elements: 8*4 bytes = 32 byt
```

```
8
```

```
32
```

```
In [53]: d = np.array([1, 2, 3, 4], dtype=np.int8)
```

```
In [48]: print(d.itemsize)
         print(d.nbytes)
```

```
1
```

```
4
```

```
In [49]: d = np.array([1, 2, 3, 4], dtype=np.int64)
```

```
In [50]: print(d.itemsize)
         print(d.nbytes)
```

```
8
```

```
32
```

```
In [54]: f = np.array(['ba', 'bb', 'c'])
         f.dtype
```

```
Out[54]: dtype('<U2')
```

```
In [55]: g = np.array(['b', 'b', 'y'])
         g.dtype
```

```
Out[55]: dtype('<U1')
```

Dimensions and shapes

```
In [56]: C = np.array([
         [1, 2, 3],
         [4, 5, 6]
         ])
```

```
In [93]: print(C.dtype)
         print(C.size)
```

```
print(C.shape)
print(C.ndim)
print(type(C[0]))
```

```
int64
6
(2, 3)
2
<class 'numpy.ndarray'>
```

```
In [64]: V = np.array([
    [
        [12, 11, 10],
        [9, 8, 7],
    ],
    [
        [6, 5, 4],
        [3, 2, 1]
    ]
])
```

```
In [67]: V.shape
```

```
Out[67]: (2, 2, 3)
```

Dimension 0 (outermost): 2 blocks (the two main arrays separated at the top level).

Dimension 1 (rows): 2 rows inside each block.

Dimension 2 (columns/innermost): 3 elements inside each row.

```
In [69]: V.ndim          #V.ndim stands for the number of dimensions (or axes) of the NumPy
          # For the array V with the shape (2, 2, 3), evaluating V.ndim returns:
```

```
Out[69]: 3
```

```
In [71]: V.size
```

```
Out[71]: 12
```

```
In [72]: C = np.array([
    [
        [12, 11, 10],
        [9, 8, 7],
    ],
    [
        [6, 5, 4]
    ]
])          #If the shape isn't consistent, it'll
```

 ValueError Traceback (most recent call last)

Cell In[72], line 1

```
----> 1 C = np.array([
      2     [
      3         [12, 11, 10],
      4         [9, 8, 7],
```

ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 1 dimensions. The detected shape was (2,) + inhomogeneous part.

```
In [91]: print(C.dtype)
         print(C.size)
         print(C.shape)
         print(type(C[0]))
```

```
int64
6
(2, 3)
<class 'numpy.ndarray'>
```

```
In [80]: a = np.array([1, 2, 3, 4])
         print(a.sum())
         print(a.mean())
         print(a.var())
```

```
10
2.5
1.25
```

```
In [89]: A = np.array([
         [1, 2, 3],
         [4, 5, 6],
         [7, 8, 9]
       ])
         print(A.sum())
         print(A.mean())
         print(A.std())
         print(A.sum(axis= 0))
         print(A.sum(axis= 1))
         print(A.mean(axis=0))
         print(A.mean(axis=1))
         print(A.std(axis=0))
         print(A.std(axis=1))
```

```
45
5.0
2.581988897471611
[12 15 18]
[ 6 15 24]
[4. 5. 6.]
[2. 5. 8.]
[2.44948974 2.44948974 2.44948974]
[0.81649658 0.81649658 0.81649658]
```

Broadcasting and Vectorized operations

```
In [110... a = np.arange(4)
print(a)
print(a+10)           #not changing "a" real value but showing what will be
print(a*10)
```

```
[0 1 2 3]
[10 11 12 13]
[ 0 10 20 30]
```

```
In [111... a += 100           #changing "a" real value
print(a)
```

```
[100 101 102 103]
```

```
In [112... [i * 10 for i in a]
```

```
Out[112... [np.int64(1000), np.int64(1010), np.int64(1020), np.int64(1030)]
```

```
In [119... a = np.arange(4)
b = np.array([10, 10, 10, 10])

print(a)
print(b)
print(a + b)
print(a * b)
```

```
[0 1 2 3]
[10 10 10 10]
[10 11 12 13]
[ 0 10 20 30]
```

Boolean

```
In [121... print(a[[0, -1]])
```

```
[0 3]
```

```
In [124... a[[True, False, False, True]]           #what we want is "True", "False we don't want
```

```
Out[124... array([0, 3])
```

```
In [125... a
```

```
Out[125... array([0, 1, 2, 3])
```

```
In [126... a[a >= 2]
```

```
Out[126... array([2, 3])
```

```
In [127... a.mean()
```

Out[127... `np.float64(1.5)`

In [128... `a[a > a.mean()]`

Out[128... `array([2, 3])`

In [130... `a[~(a > a.mean())]`

Out[130... `array([0, 1])`

The tilde `~` operator in NumPy represents the bitwise NOT operation, which acts as a logical NOT (negation) when applied to boolean arrays. It flips all True values to False and all False values to True.

In [132... `a[(a == 0) | (a == 1)]`

Out[132... `array([0, 1])`

In [134... `a[(a <= 2) & (a % 2 == 0)]` *# a % 2 == 0 even numbers are True.*

Out[134... `array([0, 2])`

In [136... `A = np.random.randint(100, size=(3, 3))`
`A`

Out[136... `array([[93, 54, 59],
[23, 58, 31],
[30, 1, 92]], dtype=int32)`

In [137... `A[np.array([
[True, False, True],
[False, True, False],
[True, False, True]
])]`

Out[137... `array([93, 59, 58, 30, 92], dtype=int32)`

In [138... `A > 30`

Out[138... `array([[True, True, True],
[False, True, True],
[False, False, True]])`

In [139... `A[A > 30]`

Out[139... `array([93, 54, 59, 58, 31, 92], dtype=int32)`

Linear Algebra

In [140... `A = np.array([
[1, 2, 3],`

```
[4, 5, 6],
[7, 8, 9]
])
```

```
In [141... B = np.array([
[6, 5],
[4, 3],
[2, 1]
])
```

```
In [142... A.dot(B)
```

```
Out[142... array([[20, 14],
[56, 41],
[92, 68]])
```

```
In [143... A @ B
```

```
Out[143... array([[20, 14],
[56, 41],
[92, 68]])
```

```
In [144... B.T
```

```
Out[144... array([[6, 4, 2],
[5, 3, 1]])
```

```
In [145... A
```

```
Out[145... array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]])
```

```
In [146... B.T @ A
```

```
Out[146... array([[36, 48, 60],
[24, 33, 42]])
```

Difference from *: The * operator performs element-wise multiplication, whereas @ calculates the dot product where rows are multiplied by columns.

Size of objects in Memory

```
In [148... # An integer in Python is > 24bytes
sys.getsizeof(1)
```

```
Out[148... 28
```

```
In [149... # Longs are even larger
sys.getsizeof(10**100)
```

```
Out[149... 72
```

```
In [150... # Numpy size is much smaller  
np.dtype(int).itemsize
```

```
Out[150... 8
```

```
In [151... # Numpy size is much smaller  
np.dtype(np.int8).itemsize
```

```
Out[151... 1
```

```
In [152... np.dtype(float).itemsize
```

```
Out[152... 8
```

```
In [154... np.dtype(float).itemsize
```

```
Out[154... 8
```

Lists are larger

```
In [155... # A one-element list  
sys.getsizeof([1])
```

```
Out[155... 64
```

```
In [156... # An array of one element in numpy  
np.array([1]).nbytes
```

```
Out[156... 8
```

Performance

```
In [157... l = list(range(100000))
```

```
In [158... a = np.arange(100000)
```

```
In [159... %time np.sum(a ** 2)
```

```
CPU times: total: 0 ns  
Wall time: 1.44 ms
```

```
Out[159... np.int64(333328333350000)
```

```
In [160... %time sum([x ** 2 for x in l])
```

```
CPU times: total: 31.2 ms  
Wall time: 20.8 ms
```

```
Out[160... 333328333350000
```

Useful Numpy functions

random

```
In [161...] np.random.random(size=2)
```

```
Out[161...] array([0.2545355 , 0.21667533])
```

```
In [162...] np.random.normal(size=2)
```

```
Out[162...] array([0.98663208, 1.24160087])
```

```
In [163...] np.random.rand(2, 4)
```

```
Out[163...] array([[0.18401529, 0.26791263, 0.30607993, 0.13125506],  
                  [0.00227013, 0.05952318, 0.27738443, 0.66283259]])
```

arange

```
In [164...] np.arange(10)
```

```
Out[164...] array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [165...] np.arange(5, 10)
```

```
Out[165...] array([5, 6, 7, 8, 9])
```

```
In [167...] np.arange(0, 1, .1)      # 0 upto 1 not included, increment of 0.1
```

```
Out[167...] array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
```

```
In [168...] np.arange(10).reshape(2, 5)
```

```
Out[168...] array([[0, 1, 2, 3, 4],  
                  [5, 6, 7, 8, 9]])
```

```
In [169...] np.arange(10).reshape(5, 2)
```

```
Out[169...] array([[0, 1],  
                  [2, 3],  
                  [4, 5],  
                  [6, 7],  
                  [8, 9]])
```

linspace

```
In [170...] np.linspace(0, 1, 5)
```

```
Out[170...] array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

```
In [171... np.linspace(0, 1, 20)
```

```
Out[171... array([0.          , 0.05263158, 0.10526316, 0.15789474, 0.21052632,
        0.26315789, 0.31578947, 0.36842105, 0.42105263, 0.47368421,
        0.52631579, 0.57894737, 0.63157895, 0.68421053, 0.73684211,
        0.78947368, 0.84210526, 0.89473684, 0.94736842, 1.          ])
```

```
In [172... np.linspace(0, 1, 20, False)
```

```
Out[172... array([0.  , 0.05, 0.1  , 0.15, 0.2  , 0.25, 0.3  , 0.35, 0.4  , 0.45, 0.5  ,
        0.55, 0.6  , 0.65, 0.7  , 0.75, 0.8  , 0.85, 0.9  , 0.95])
```

zeros

```
In [173... np.zeros(5)
```

```
Out[173... array([0., 0., 0., 0., 0.])
```

```
In [174... np.zeros((3, 3))
```

```
Out[174... array([[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]])
```

```
In [176... np.zeros((3, 3), dtype=int)
```

```
Out[176... array([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

ones

```
In [177... np.ones(5)
```

```
Out[177... array([1., 1., 1., 1., 1.])
```

```
In [178... np.ones((3, 3))
```

```
Out[178... array([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
```

empty, identity and eye

```
In [180... np.empty(5)
```

```
Out[180... array([1., 1., 1., 1., 1.])
```

```
In [181... np.empty((2, 2))
```

```
Out[181... array([[0.25, 0.5 ],
        [0.75, 1.   ]])
```

```
In [182... np.identity(3)
```

```
Out[182... array([[1., 0., 0.],  
        [0., 1., 0.],  
        [0., 0., 1.]])
```

```
In [183... np.eye(3, 3)
```

```
Out[183... array([[1., 0., 0.],  
        [0., 1., 0.],  
        [0., 0., 1.]])
```

```
In [184... np.eye(8, 4)
```

```
Out[184... array([[1., 0., 0., 0.],  
        [0., 1., 0., 0.],  
        [0., 0., 1., 0.],  
        [0., 0., 0., 1.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.]])
```

```
In [185... np.eye(8, 4, k=1)
```

```
Out[185... array([[0., 1., 0., 0.],  
        [0., 0., 1., 0.],  
        [0., 0., 0., 1.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.]])
```

```
In [186... np.eye(8, 4, k=-3)
```

```
Out[186... array([[0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [0., 0., 0., 0.],  
        [1., 0., 0., 0.],  
        [0., 1., 0., 0.],  
        [0., 0., 1., 0.],  
        [0., 0., 0., 1.],  
        [0., 0., 0., 0.]])
```

```
In [187... "Hello World"[6]
```

```
Out[187... 'W'
```

```
In [ ]:
```